

Documentación: Data Warehouse FleetLogix

Introducción al Proyecto

El proyecto FleetLogix Data Warehouse representa la evolución estratégica de la empresa desde un enfoque operativo transaccional hacia una visión analítica de largo plazo. Mientras que la base de datos PostgreSQL existente maneja las operaciones diarias, este Data Warehouse en Snowflake está diseñado específicamente para:

- Evaluar desempeño histórico de la flota y operaciones
- Identificar tendencias en entregas, eficiencia y costos
- Comparar indicadores clave en múltiples niveles de granularidad
- Soportar decisiones estratégicas con datos consolidados y optimizados

Arquitectura del Data Warehouse

Modelo Estrella Implementado

El código Snowflake implementa un modelo dimensional estrella compuesto por:

Tablas de Dimensiones (Descriptores contextuales)

1. `dim_date` - Calendario completo para análisis temporal
2. `dim_time` - Desglose horario para análisis por turnos
3. `dim_vehicle` - Flota vehicular con SCD Type 2
4. `dim_driver` - Conductores con seguimiento histórico
5. `dim_route` - Rutas y características geográficas
6. `dim_customer` - Clientes y su segmentación

Tabla de Hechos (Métricas y eventos)

`fact_deliveries` - Eventos de entrega con:

- Claves foráneas hacia todas las dimensiones
- Métricas operativas: peso, distancia, tiempo, combustible
- Indicadores de calidad: puntualidad, daños, firmas
- Métricas calculadas: eficiencia, costos, revenue

Características Técnicas Destacadas

1. SCD Type 2 (Slowly Changing Dimensions)

```
valid_from DATE,  
valid_to DATE,  
is_current BOOLEAN
```

Permite rastrear cambios históricos en vehículos y conductores manteniendo la integridad temporal.

2. Configuración Snowflake Optimizada

- Time Travel: 30 días de retención para recuperación de datos
- Warehouse escalable: Configuración XSMALL con auto-suspensión
- Estructura modular: Database → Schema → Tablas

3. Seguridad y Gobernanza

- Vistas seguras por rol de negocio
- Roles especializados: SALES_ANALYST, OPERATIONS_ANALYST
- Control de acceso granular a nivel de vista

4. Pipeline ETL/ELT Preparado

- Tabla de staging para carga incremental
- Campos de auditoría: ETL batch y timestamps
- Estructura flexible para procesos automatizados

Flujo de Datos Propuesto

PostgreSQL (Transaccional)

- Python ETL Pipeline
- Snowflake Staging
- Modelo Estrella
- Vistas Analíticas

Beneficios del Diseño

Para Negocio

- Consultas 10-100x más rápidas que en PostgreSQL
- Capacidad de análisis histórico multi-dimensional
- Segmentación avanzada de clientes y rutas
- Monitoreo de tendencias y KPIs en tiempo cuasi-real

Documentación Completa: Pipeline ETL FleetLogix

Arquitectura del Pipeline ETL

El sistema implementa un pipeline ETL completo que migra datos desde PostgreSQL (transaccional) hacia Snowflake (Data Warehouse analítico), siguiendo el modelo estrella previamente definido.

1. MA_extract.py - Extracción de Datos

Propósito Principal

Extraer datos de la base transaccional PostgreSQL y prepararlos para el proceso ETL.

Características Clave

- Conexión dual: PostgreSQL (origen) y Snowflake (destino)
- Extracción por fecha: Selecciona la fecha más reciente con datos disponibles
- Generación de claves dimensionales: Pre-calcula claves para el modelo estrella
- Formato optimizado: Guarda en Parquet para procesamiento eficiente

Flujo de Ejecución

1. Verificar conectividad Snowflake
2. Buscar fechas disponibles en PostgreSQL
3. Extraer datos de la fecha más reciente
4. Generar claves dimensionales (date_key, time_key)
5. Guardar en formato Parquet

Relación con Otros Módulos

- Input: Configuración de conexión (settings.ini)
- Output: Archivo Parquet para MA_transform.py
- Dependencias: Estructura de tablas PostgreSQL

2. MA_transform.py - Transformación de Datos

Propósito Principal

Aplicar reglas de negocio, validaciones y preparar datos para el modelo dimensional.

Transformaciones Aplicadas

1. Cálculo de métricas temporales

`delivery_duration_minutes = delivered_datetime - scheduled_datetime`

`delay_minutes_calculated = mismo cálculo con validación`

2. Métricas de eficiencia

`fuel_efficiency_calculated = distance_km / fuel_consumed`

3. Validaciones de calidad

- Duración entre 1 minuto y 24 horas

- Distancia entre 0 y 5000 km

- Peso entre 0 y 10000 kg

- Eficiencia entre 0 y 50 km/lt

4. Score de calidad (0-100 puntos)

Preparación para Snowflake

- Columnas en MAYÚSCULAS (estándar Snowflake)
- Selección de columnas finales según modelo estrella
- Redondeo de valores numéricos
- Mantenimiento de booleanos nativos

Relación con Otros Módulos

- Input: Archivo Parquet de MA_extract.py
- Output: DataFrame transformado para MA_load.py
- Validación: Compatibilidad con estructura Snowflake

3. MA_load.py - Carga a Snowflake

Propósito Principal

Cargar los datos transformados en las tablas del Data Warehouse Snowflake.

Características Técnicas

- # Manejo robusto de Snowflake
- Nombres en MAYÚSCULAS (sin comillas)
- Creación automática de tablas
- Carga incremental (APPEND)
- Verificación pre/post carga

Mecanismos de Seguridad

- Verificación de conexión antes de operaciones críticas
- Conteo de registros para validar integridad
- Manejo de errores con logging detallado
- Disposición de conexiones para evitar leaks

Flujo de Carga

1. Verificar conexión Snowflake
2. Contar registros existentes
3. Crear tabla si no existe
4. Cargar datos transformados
5. Verificar conteo final

Relación con Otros Módulos

- Input: DataFrame transformado de MA_transform.py
 - Output: Datos en tablas Snowflake
 - Dependencias: Estructura DW definida en script SQL
-

4. MA_main.py - Orquestador Principal

Propósito Principal

Coordinar la ejecución secuencial de los 3 módulos y proveer monitoreo integral.

Características de Orquestación

Gestión de flujo completo

FASE 1: Verificaciones iniciales

FASE 2: Extracción de PostgreSQL

FASE 3: Transformación de datos

FASE 4: Carga a Snowflake

Mecanismos de Control

- Logging estructurado con timestamps
- Manejo de excepciones con traceback completo
- Modo prueba para validaciones rápidas
- Métricas de ejecución (tiempo, volumen, eficiencia)

Argumentos de Línea de Comandos

```
python MA_main.py --limit 10000    # Límite de registros
python MA_main.py --test            # Modo prueba
python MA_main.py --verbose         # Logging detallado
```

Flujo de Datos Integrado

Secuencia de Ejecución

MA_extract.py (PostgreSQL → Parquet)

↓

MA_transform.py (Parquet → DataFrame transformado)

↓

MA_load.py (DataFrame → Snowflake DW)

↓

MA_main.py (verifica y reporta)

Dependencias entre Módulos

```
MA_extract.py
  ↓ (produce)
MA_transform.py
  ↓ (consume)
MA_load.py
  ↑ (orquestra)
MA_main.py
```

Archivos de Configuración

```
config/
├── settings.ini          # Credenciales BD y Snowflake

data/
├── staging/              # Archivos Parquet temporales

logs/
├── etl_pipeline.log      # Logs de ejecución
```

Características de Robustez

Manejo de Errores

- Reconexión automática en fallos de conexión
- Validación de datos en cada fase
- Logging detallado para troubleshooting
- Verificación de integridad post-carga

Escalabilidad

- Límites configurables para control de volumen
- Carga incremental para datasets grandes
- Procesamiento por lotes optimizado
- Formato Parquet para eficiencia I/O

Mantenibilidad

- Configuración centralizada en settings.ini
- Logging estructurado para monitoreo
- Código modular con responsabilidades separadas
- Documentación integrada en cada módulo

Beneficios del Diseño

1. Separación de responsabilidades: Cada módulo tiene un propósito específico
2. Reutilización: Módulos pueden ejecutarse independientemente
3. Monitoreo: Logging completo y métricas de ejecución
4. Escalabilidad: Preparado para grandes volúmenes de datos
5. Mantenibilidad: Código claro y documentado

Este pipeline representa una implementación profesional y lista para producción del proceso ETL requerido para alimentar el Data Warehouse FleetLogix.

Conclusión Final

El Data Warehouse FleetLogix no es solo un proyecto técnico exitoso, sino una capacidad estratégica habilitada que posiciona a la empresa para:

"Dejar de operar en modo reactivo y comenzar a dirigir el negocio basado en datos históricos, tendencias predictivas y métricas de performance en tiempo real."

La implementación cumple con los más altos estándares de ingeniería de datos mientras entrega valor de negocio inmediato y sienta las bases para una organización data-driven en los próximos años.